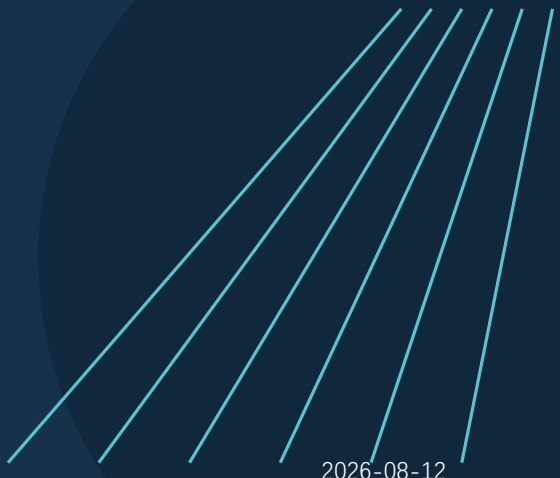
PyRepair Agent

十天实战学习手册

从会运行，到能解释、能调试、能改进、能面试

10天 × 每天4-6小时 | Python / pytest / Agent开发

实战路线 A



PYREPAIR LEARNING GUIDE / VERSION 1.0

2026-08-12

目录

建议按天执行，不要跳过动手任务和完成标准。阅读源码时始终把实现、测试和一次真实运行证据放在一起。

如何使用这本手册 3

项目全景 4

开始前：环境与命令速查 5

第1天：从使用者视角跑通项目 6

第2天：CLI、配置与总协调器 7

第3天：pytest执行与失败解析 8

第4天：临时工作区与五个受限工具 10

第5天：DeepSeek SDK与Tool Calling循环 11

第6天：Policy、Review与全量回归 12

第7天：产物、可审计性与故障定位 14

第8天：测试体系与FakeLLM 15

第9天：Benchmark与数据表达 17

第10天：亲手完成pytest 9解析改进 18

核心模块阅读地图 20

Git最小知识：只学本项目立即需要的 21

面试讲解模板 22

简历写法与证据对应 23

常见错误与排查手册 23

十天验收清单 24

后续30天进阶路线 25

结语 26

学习目标

这不是一本要求你背源码的书。完成后，你应该能独立演示 PyRepair、画出完整调用链、解释安全设计、定位一次失败发生在哪一层，并亲手完成一个带测试的改进。

如何使用这本手册

最重要的学习原则

不要从 `src/pyrepair` 的第一个文件开始逐行阅读。正确方法是跟随一次 CLI 请求：参数如何进入、pytest 如何运行、失败如何结构化、模型如何调用工具、补丁如何被拦截和验证、结果如何落盘。每读一个源码文件，都同时阅读对应测试。

每天建议准备一张纸或一个 Markdown 笔记，固定记录四件事：

1. 这个模块接收什么输入？
2. 它返回什么输出？
3. 为什么必须独立存在？
4. 它失败后，上层如何处理？

十天结束时的四级目标

等级	你能做到什么	验收方式
L1 会使用	安装、测试、dry-run、真实调用、查看产物	不看提示完成一次演示
L2 会解释	画出调用链并解释每个安全门	5 分钟口述项目架构
L3 会调试	根据阶段和产物判断失败层级	分析一个失败 `report.json`
L4 会改进	修改解析器、补测试、跑回归和 benchmark	用数据说明修改前后差异

你现在不必先学完的内容

- 不必先学完整的多 Agent 框架；本项目只有 Repair 与 Review 两个清晰角色。
- 不必先学向量数据库、RAG、长期记忆或 MCP；MVP 没有使用它们。
- 不必熟练掌握 Docker、Kubernetes 或 Java。
- 不必马上学 GitHub Pull Request；前 8 天只需要本地 Git 的最小操作。
- 不必理解 OpenAI SDK 的全部接口；先理解 messages、tools、tool calls 和 usage。

学习规则

每天至少 60% 时间要花在运行命令、打断点、改小实验和回答自测题上。只阅读会产生“看懂了”的错觉。

项目全景

PyRepair 解决什么问题

输入是一个能够复现 pytest 失败的 Python 项目；输出是一个经过安全检查、独立 Review 和测试验证的候选补丁。它不会自动把补丁应用到原仓库，也不会自动 Commit、Push 或创建 Pull Request。

项目刻意选择狭窄场景：单 Bug、小补丁、Python 源码、pytest。狭窄不是缺点，而是使安全边界和评测指标能够被明确验证。

完整调用链

```
开发者运行 pyrepair fix
    |
    v
CLI 解析 repo / target / model / dry-run
    |
    v
WorkspaceManager 创建 baseline 与 working 临时副本
    |
    v
TestRunner 运行 pytest，PytestParser 生成 FailureReport
    |
    v
RepairAgent 与 DeepSeek 进行 Tool Calling 循环
    |
    v
WorkspaceSession.snapshot() 生成 PatchSnapshot
    |
    v
PatchPolicyGate -> ReviewAgent -> Verifier
    |
    v
RunRecorder 写出 fix.patch / report.json / trace.jsonl / pytest.log
```

三种责任不能混在一起

责任	由谁完成	为什么
提出可能的修复	LLM Agent	需要代码语义推理，允许一定概率性
限制允许修改的边界	Python Policy	路径、行数、文件数量必须确定可重复
判断修复是否有效	pytest + Verifier	不能相信模型自己说“修好了”

关键数据对象

- `RepairRequest`：一次修复请求，包含仓库、目标测试、模型开关和限制。
- `TestRun`：一次 pytest 子进程的命令、退出码、标准输出、错误输出和耗时。
- `FailureReport`：从 pytest 文本提取的 node id、异常、摘要和相关路径。
- `PatchSnapshot`：working 副本相对 baseline 的文件变化集合。
- `ModelTurn`：模型一轮回复，包括文本、推理、工具调用和 Token。
- `VerificationResult`：目标测试、全量测试、策略复查和新增回归的汇总。
- `RunResult`：CLI 最终看到的状态、尝试次数、原因和产物目录。

开始前：环境与命令速查

第一次创建虚拟环境

在 PowerShell 中执行：

```
cd "D:\Project\PyRepair Agent"

& "D:\Anaconda 3\python.exe" -m venv .venv

.\.venv\Scripts\python.exe -m pip install -e ".[dev]"
```

逐段理解：

- `cd`：把当前目录切换到项目根目录。
- `&`：PowerShell 的调用运算符，用来运行路径中带空格的程序。
- `python.exe -m venv .venv`：让指定 Python 创建隔离环境。
- `-m pip`：使用当前这个 Python 对应的 pip，避免装到别的环境。
- `-e`：editable install，源码修改后通常不需要重新安装。
- `.[dev]`：安装当前项目及 pytest、coverage、Ruff、build 等开发依赖。

三条最常用验证命令

```
.\.venv\Scripts\python.exe -m pytest -q

.\.venv\Scripts\python.exe -m pytest --cov=pyrepair --cov-report=term-missing -q

.\.venv\Scripts\python.exe -m ruff check src tests benchmarks
```

`-q` 表示减少 pytest 的冗余输出。`--cov=pyrepair` 收集包覆盖率，`term-missing` 显示未覆盖行。Ruff 做静态质量检查，但它不能替代测试。

两种运行方式

```
# 只诊断，不调用 LLM
.\.venv\Scripts\pyrepair.exe fix --repo examples/buggy_bank --dry-run

# 调用 DeepSeek 生成并验证候选补丁
Set-Item -Path Env:DEEPSEEK_API_KEY -Value "你的 API Key"
.\.venv\Scripts\pyrepair.exe fix --repo examples/buggy_bank --model deepseek-v4-flash
```

`pyrepair` 是 `pyproject.toml` 中注册的 CLI 命令；`fix` 是子命令；`--repo` 是待诊断项目；`--dry-run` 会停在诊断阶段；`--model` 覆盖默认模型。

> 安全提醒
> API Key 只放在当前 PowerShell 环境变量，不写进代码、`.env`、README、截图或 Git 提交。

第1天：从使用者视角跑通项目

今日目标

不读核心实现，先建立“输入、阶段、输出”的直觉。你要亲手完成安装、完整测试、失败复现、dry-run 和产物查看。

时间安排（4-6小时）

时间	内容
45 分钟	重建 `.venv`，理解每一段命令
45 分钟	跑完整测试、覆盖率和 Ruff
60 分钟	阅读 README 的使用、安全边界和局限
90 分钟	对 `examples/buggy_bank` 先运行 pytest，再运行 dry-run
60 分钟	查看一次真实运行留下的四个产物
30 分钟	画第一版流程图并回答自测题

实操一：先看原始失败

```
.\.venv\Scripts\python.exe -m pytest examples/buggy_bank -q
```

记录三个信息：失败测试的 node id、异常或断言摘要、pytest 退出码。PyRepair 的 baseline 做的正是这一步，但它会把结果包装成 `TestRun`。

实操二：只诊断

```
.\.venv\Scripts\pyrepair.exe fix --repo examples/buggy_bank --dry-run
```

观察终端阶段。dry-run 不调用 LLM，也不生成候选修改；它用于验证项目路径、pytest 是否可启动以及失败输出能否解析。

实操三：查看真实运行产物

完成一次真实模型调用后：

```
$runDir = Get-ChildItem examples/buggy_bank/.pyrepair/runs -Directory |
    Sort-Object LastWriteTime -Descending |
    Select-Object -First 1

Get-Content "$($runDir.FullName)/fix.patch"
Get-Content "$($runDir.FullName)/report.json"
Get-Content "$($runDir.FullName)/trace.jsonl"
Get-Content "$($runDir.FullName)/pytest.log"
```

`$runDir` 是变量；管道 `|` 把左边输出交给右边；排序后取最新目录。四个文件分别回答：改了什么、最终结论是什么、过程发生了什么、测试证据是什么。

自测问题

1. 为什么必须先运行原始 pytest，而不是直接问模型？
2. dry-run 为什么不需要 API Key？
3. 为什么生成的是候选补丁，而不是直接修改原仓库？
4. `report.json` 与 `pytest.log` 的用途有什么不同？

完成标准

- 你可以不复制粘贴，独立运行三个验证命令。
- 你能用自己的话解释 `pyrepair fix --repo ... --dry-run` 每一段。
- 你画出至少包含 baseline、agent、policy、review、verify、artifacts 的流程图。

学习记录：今天最不理解的命令是 _____；我准备通过 _____ 验证自己的理解。

第2天：CLI、配置与总协调器

今日目标

理解请求如何从命令行进入系统，以及 `RepairCoordinator` 为什么是主流程而不是“万能类”。今天先抓骨架，不钻进每个细节。

源码路线

```
pyproject.toml
  -> src/pyrepair/cli.py
  -> src/pyrepair/config.py
  -> src/pyrepair/models.py
  -> src/pyrepair/coordinator.py
```

同时阅读：`tests/test_cli.py`、`tests/test_config.py`、`tests/test_coordinator.py`。

时间安排

时间	内容
45 分钟	找到 `pyrepair` CLI 入口
60 分钟	阅读 `fix()` 参数和错误退出码
45 分钟	阅读 `AppConfig` 与 `load_config()`
90 分钟	只读 `RepairCoordinator.run()` 的阶段骨架
60 分钟	对照协调器测试理解提前退出分支
30 分钟	画请求对象和返回对象

关键问题一：CLI入口在哪里

`pyproject.toml` 中的：

```
[project.scripts]
pyrepair = "pyrepair.cli:app"
```

意思是安装项目后，命令 `pyrepair` 会调用 `pyrepair.cli` 模块中的 Typer 应用。`fix()` 负责把字符串和开关转换为 `RepairRequest`，捕获用户可理解的配置错误，并用退出码表达结果。

关键问题二：Coordinator负责什么

`RepairCoordinator` 只负责组织协作者：Workspace、Runner、Parser、RepairAgent、Policy、ReviewAgent、Verifier、Recorder。它不应该自己实现文件搜索、pytest 解析或模型 HTTP 细节。

阅读 `run()` 时，用不同颜色标出：

- 正常阶段：baseline、attempt、policy、review、verify。
- 提前退出：项目已通过、输出不可解析、无补丁、策略拒绝、Review拒绝、验证失败。
- finally清理：无论成功失败都要释放临时工作区。

动手实验

```
.\.venv\Scripts\pyrepair.exe --help
.\.venv\Scripts\pyrepair.exe fix --help
.\.venv\Scripts\python.exe -m pytest tests/test_cli.py tests/test_config.py -v
.\.venv\Scripts\python.exe -m pytest tests/test_coordinator.py -v
```

给 `tests/test_coordinator.py` 中每个测试写一句中文标题，例如“没有生成补丁时最多尝试三次”“策略拒绝后不会进入Review”。这会比逐行翻译实现更快建立状态机。

自测问题

1. 为什么配置错误应该返回退出码2，而普通修复失败返回1？
2. `build_coordinator()` 为什么需要组装依赖？
3. 如果 baseline 已经通过，应该调用 LLM 吗？
4. 为什么 Coordinator 的阶段需要被 Recorder 记录？

完成标准

你能在白纸上写出 `fix() -> RepairRequest -> coordinator.run() -> RunResult`，并口述至少四条提前退出路径。

学习记录：我最容易混淆的提前退出路径是 _____；对应测试是 _____。

第3天：pytest执行与失败解析

今日目标

理解非 LLM 的第一条核心数据链：pytest 文本如何变成结构化 `FailureReport`。这一天直接关系到 benchmark 中两个失败案例。

建议时间分配

45分钟复现不同pytest输出；75分钟读Runner和数据对象；75分钟读Parser及fixtures；60分钟运行测试并画数据流；45分钟整理失败类型；30分钟完成自测。

源码路线

```
src/pyrepair/testing.py
  -> TestRun（models.py）
  -> src/pyrepair/pytest_parser.py
  -> FailureReport（models.py）
```

对应测试：`test_testing.py`、`test_pytest_parser.py` 和 `tests/fixtures/pytest_outputs/`。

TestRunner必须掌握的设计

- 使用参数列表启动 `sys.executable -m pytest`，不使用 `shell=True`。
- 固定 `-q --tb=short --disable-warnings`，减少输出变化。

- 支持已识别的目标 node id，但不接受任意 Shell 命令。
- 捕获 stdout、stderr、退出码和耗时。
- 将超时和无法启动转换为 `TestRun`，而不是让异常穿透整个程序。

Parser为什么要保守

解析器不能“猜一个可能的测试”。如果没有可靠识别到失败 node id，就返回不可解析状态，让协调器停止。这会降低覆盖范围，却避免 Agent 在错误上下文中乱改代码。

当前解析器重点识别类似：

```
FAILED tests/test_bank.py::test_withdraw - AssertionError: ...
```

而 pytest 9 有时只输出：

```
FAILED tests/test_bank.py::test_withdraw
```

这正是 `case_05` 和 `case_08` 在调用 LLM 前失败的原因。

动手实验

```
.\.venv\Scripts\python.exe -m pytest tests/test_testing.py -v
.\.venv\Scripts\python.exe -m pytest tests/test_pytest_parser.py -v

Get-Content tests/fixtures/pytest_outputs/assertion.txt
Get-Content tests/fixtures/pytest_outputs/exception.txt
Get-Content tests/fixtures/pytest_outputs/unparseable.txt
```

接着在 Python 断点或临时 REPL 中观察 `PytestParser().parse(run)` 的输出字段。不要提交临时打印；实验结束后还原。

易混淆概念

概念	含义
collection error	pytest 在收集测试时就失败，测试函数还没运行
test failure	测试运行了，但断言或代码异常导致失败
timeout	子进程超过限定时间
launch error	Python 或 pytest 子进程无法启动
unparseable	pytest有输出，但当前规则无法可靠提取故障

自测问题

1. 为什么使用 `sys.executable` 而不是硬编码 `python`?
2. 为什么不用 `shell=True`?
3. 输出不可解析为什么不直接把全部文本交给 LLM?
4. Parser失败是否等于DeepSeek能力不行?

完成标准

你能够解释 `TestRun -> FailureReport` 每个字段的来源，并准确说明 case05、case08 属于基础设施解析失败，不是模型修复失败。

学习记录：我今天见到的三种pytest失败形态是 _____、_____、_____。

第4天：临时工作区与五个受限工具

今日目标

理解 PyRepair 最重要的安全基础：LLM 不接触原仓库，也没有任意文件和任意命令权限。

建议时间分配

60分钟理解baseline/working；90分钟阅读Workspace和测试；90分钟阅读五个工具；60分钟整理攻击输入与防御；30分钟画权限边界；30分钟自测。

源码路线

```
src/pyrepair/workspace.py
  -> WorkspaceSession
  -> src/pyrepair/tools.py
  -> ToolRegistry
```

对应测试：`test_workspace.py` 和 `test_tools.py`。

baseline与working副本

WorkspaceManager 会检查仓库存在且包含 Python 源码，然后创建临时根目录。baseline 保留初始状态，working 供 Agent 修改。`snapshot()` 比较两者并生成 `PatchSnapshot`；失败尝试可以通过 `restore_working_from_baseline()` 丢弃，再从干净状态开始。

这种设计解决两个问题：

1. 模型或工具失败不会污染用户原仓库。
2. Diff 不依赖用户是否使用 Git，因此普通文件夹也能运行。

五个工具

工具	能做什么	不能做什么
`read_file`	读取工作区内 UTF-8 文本	不能越界、不能读超大或二进制文件
`search_code`	在 Python 文件中搜索文本	不能扫描无限文件或任意外部目录
`replace_text`	唯一匹配后精确替换源码	不能改测试、不能模糊批量替换
`show_diff`	查看 baseline 到 working 的变化	只读，不能自行批准修改
`run_target_test`	运行解析器已识别的目标测试	不能注入任意 pytest 参数或 Shell 命令

路径安全

`resolve_inside()` 会解析用户路径并确认最终路径仍在工作区根目录内。因此 `../secret.txt`、绝对路径和 Windows 盘符逃逸都会被拒绝。只检查字符串中有没有 `..` 不够，因为符号链接和规范化路径可能绕过简单判断。

动手实验

```
.\.venv\Scripts\python.exe -m pytest tests/test_workspace.py -v
.\.venv\Scripts\python.exe -m pytest tests/test_tools.py -v
```

任选三个安全测试，写出“攻击输入 -> 防御代码 -> 预期错误”三列表。然后回答：如果给 Agent 一个通用 `run_shell(command)`，它可能绕过哪些现有边界？

自测问题

1. 临时工作区和 Git branch 是同一个概念吗？
2. 为什么 `replace_text` 要求旧文本只出现一次？
3. 为什么 `run_target_test` 不能由模型自由填写测试命令？
4. Agent修改working后，原项目为什么仍然不变？

完成标准

你可以完整说出五个工具及其限制，并能解释“能力更少”如何换来更可验证的安全性。

学习记录：如果只能保留三个工具，我会保留 _____，原因是 _____。

第5天：DeepSeek SDK与Tool Calling循环

今日目标

理解 Agent 的最小闭环，不追求学完整 SDK。今天重点是消息如何流动，以及模型为什么不能直接调用本地函数。

建议时间分配

60分钟补SDK最小概念；90分钟读DeepSeekClient；90分钟跟踪RepairAgent消息循环；60分钟对照FakeLLM测试；45分钟画消息序列；30分钟口述练习。

源码路线

```
src/pyrepair/deepseek.py
  -> ModelClient Protocol
  -> DeepSeekClient.complete()
  -> ModelTurn
  -> src/pyrepair/repair_agent.py
```

对应测试：`test_deepseek.py`、`test_repair_agent.py`、`tests/fakes.py`。

Anthropic API、Claude SDK与本项目的关系

SDK 是调用某家公司 API 的客户端库，不等于模型本身。PyRepair 使用 OpenAI Python SDK 连接 DeepSeek 的 OpenAI-compatible API。核心原因是请求和 Tool Calling 数据结构兼容，而不是在调用 OpenAI 模型。

如果未来 DeepSeek V4 Pro 正式发布且 API 标识可用，通常只需把允许模型列表和默认/命令行配置调整为官方模型名；Agent循环、安全门和测试架构不需要重写。

Tool Calling循环

1. Python发送 system + user messages 和工具 schemas
2. 模型返回普通文本，或一个/多个 tool call
3. Python检查工具名与JSON参数
4. ToolRegistry在本地执行受限工具
5. Python把工具结果作为tool message追加回上下文
6. 回到步骤2，直到模型结束或达到调用上限

模型只会返回“请求调用某工具及参数”；真正执行文件读取、替换和测试的是 Python。这个分离点是理解 Agent 开发的关键。

为什么需要调用上限

没有上限时，模型可能重复搜索、重复运行测试或陷入无效循环，造成费用和时间失控。`RepairAgent` 在每次执行前检查上限，不能因为最后一轮又返回工具调用就多执行一次。

FakeLLM不是低准确率版本

`FakeLLM` 接收预先安排的 `ModelTurn`，用于稳定测试 Python 控制逻辑。它不评测模型能否修复真实 Bug，所以不存在“用 FakeLLM 导致模型准确率降低”。离线单元测试回答的是“我们的程序面对某种模型回复时行为是否正确”；live test 和 benchmark 才回答真实兼容性与端到端效果。

动手实验

```
.\.venv\Scripts\python.exe -m pytest tests/test_repair_agent.py -v
.\.venv\Scripts\python.exe -m pytest tests/test_deepseek.py -v
```

选择 `test_agent_executes_tools_and_returns_results_to_model`，在纸上画每一条 message 的 role：system、user、assistant、tool、assistant。再找出“未知工具”和“超过调用上限”的测试。

自测问题

1. 模型返回tool call后，谁真正执行函数？
2. `ModelClient` Protocol 有什么价值？
3. 为什么无工具的 Review 调用要省略 tools 参数？
4. SDK重试和业务层重试有什么区别？

完成标准

你能用不超过两分钟讲清 Tool Calling 循环，并画出一轮 `read_file -> tool result -> replace_text` 的消息序列。

学习记录：一轮Tool Calling中由模型决定的是 _____，由Python决定的是 _____。

第6天：Policy、Review与全量回归

今日目标

理解三道安全门各自解决什么问题。面试时这部分最能体现项目不是简单套壳。

建议时间分配

75分钟读Policy及参数化测试；60分钟读Review；75分钟读Verifier；60分钟制作风险-防线-证据表；45分钟面试口述；30分钟自测。

源码路线

```
src/pyrepair/policy.py
  -> src/pyrepair/review_agent.py
  -> src/pyrepair/verifier.py
  -> VerificationResult.passed
```

对应测试：`test_policy.py`、`test_review_agent.py`、`test_verifier.py`。

第一层：确定性Policy

Policy用普通Python规则检查：

- 只能修改允许的 Python 源码路径。
- 禁止测试、CI、密钥、`.env` 和依赖锁文件。
- 最多修改3个文件、80行增删。
- 禁止新增 `skip`、`xfail`。
- 禁止删除或弱化已有断言。

这些条件明确、可重复，不应该交给概率模型判断。

第二层：独立只读Review

Review Agent看到故障、候选Diff和证据，但没有修改工具。它评估根因是否匹配、修改是否最小、是否可能隐藏风险，并返回结构化 JSON：`approved`、`risk`、`findings`、`reason`。

“独立”表示它使用新上下文，不继承 Repair Agent 为自己补丁辩护的对话；“只读”表示即使发现问题也只能拒绝，不能偷偷改补丁。

第三层：Verifier

Verifier依次运行：

1. 目标测试：原来的失败是否修复？
2. 全量测试：项目其他测试是否仍通过？
3. 新增回归比较：全量测试的新失败是否不在baseline失败集合中？
4. Policy复查：测试运行产生的字节码或其他变化不能改变安全结论。

全量测试回归就是“修完目标测试后，再运行整个测试套件”。只跑目标测试可能把局部问题修好，却破坏其他功能。

为什么三层不能互相替代

只保留某一层	会漏掉什么
只有Policy	合法路径中的逻辑错误和过度修复
只有Review	模型可能不稳定地放过明确违规
只有pytest	修改测试、删除断言也可能让测试变绿

动手实验

```
.\.venv\Scripts\python.exe -m pytest tests/test_policy.py -v
.\.venv\Scripts\python.exe -m pytest tests/test_review_agent.py -v
.\.venv\Scripts\python.exe -m pytest tests/test_verifier.py -v
```

从每个文件选两个测试，整理成“风险 -> 防线 -> 证据”。例如“新增pytest.skip -> Policy -> rejected reason”。

自测问题

1. 为什么Review通过后还要跑测试？

2. 为什么测试全通过后还要检查Policy?
3. baseline本来就失败的测试算新增回归吗?
4. `VerificationResult.passed` 为什么必须由多个布尔条件共同决定?

完成标准

你能回答面试追问："为什么已经有Claude Code一类Agent，还要做PyRepair？"重点不是更通用，而是针对pytest修复提供可复现、可审计、确定性安全边界和量化评测。

学习记录：最适合用确定性规则处理的风险是 _____；最需要语义Review的是 _____。

第7天：产物、可审计性与故障定位

今日目标

把"程序运行过"升级为"运行过程可以复查"。学会不看源码，仅根据阶段、退出码和产物判断失败发生在哪一层。

建议时间分配

60分钟读Recorder；75分钟对照四类产物；90分钟分析三种失败运行；60分钟制作故障决策树；45分钟脱敏与原子写入；30分钟自测。

源码路线

```
src/pyrepair/recorder.py
  -> RunRecorder
  -> coordinator._finish()
  -> .pyrepair/runs/<run-id>/
```

对应测试：`test_recorder.py` 和 `test_coordinator.py` 中与记录相关的测试。

四个产物

文件	主要内容	调试问题
`fix.patch`	候选修改的统一Diff	具体改了什么?
`report.json`	最终状态、原因、Review、验证汇总	为什么成功或失败?
`trace.jsonl`	每行一个事件、工具调用、Token	过程在哪一步偏离?
`pytest.log`	baseline、目标、全量pytest证据	测试真实输出是什么?

JSONL是一行一个JSON对象，适合边运行边追加；即使程序中途失败，前面的事件仍可保留。报告适合读取最终状态，trace适合还原时间线。

敏感信息处理

Recorder在写盘前递归脱敏敏感字段和当前API Key。原子写入使用临时文件后替换目标，避免进程中断留下半个JSON。

故障定位决策树

```
没有进入 attempt
  -> 检查 baseline、项目路径、pytest启动和解析

进入 attempt 但没有 patch
  -> 检查 tool call、调用上限、replace_text结果

停在 policy
  -> 检查修改路径、行数、文件数、skip/断言

停在 review
  -> 查看 findings、risk、reason

停在 verify
  -> 区分目标仍失败、全量回归失败、新增回归
```

动手任务

选择一个已有测试，通过 FakeLLM 人为构造以下三个结果并阅读测试断言：无补丁、Policy拒绝、Review拒绝。然后为每种情况写出你期望在 `report.json` 中看到的关键字段。

```
.\.venv\Scripts\python.exe -m pytest tests/test_recorder.py tests/test_coordinator.py -v
```

自测问题

1. 为什么同时需要 `report.json` 和 `trace.jsonl`?
2. 为什么失败运行也必须生成产物?
3. 原子写入解决什么问题?
4. 如果终端只显示 `failed`，下一步优先看哪个文件?

完成标准

拿到任意一次运行目录，你能在五分钟内指出最终状态、失败层级、候选改动、Review意见和测试证据。

学习记录：我判断一次运行失败层级时首先查看 _____，然后查看 _____。

第8天：测试体系与FakeLLM

今日目标

理解“普通测试不调用LLM”为什么反而让工程验证更可靠，以及四种测试分别回答什么问题。

建议时间分配

60分钟梳理四层测试；90分钟阅读FakeLLM和Agent测试；90分钟阅读Coordinator测试；60分钟运行覆盖率；45分钟设计一个离线案例；30分钟自测。

四层测试

层次	是否联网	是否花Token	回答的问题
单元测试	否	否	一个函数或规则是否按预期工作?
协调器离线测试	否	否	给定模型回复，完整控制流是否正确?
Live smoke	是	是	鉴权、消息、Thinking和Tool Calling是否兼容?
Benchmark	是	是	固定真实Bug集合上的端到端效果如何?

如果所有普通测试都调用真实LLM，会出现网络波动、模型输出随机、费用增加、难以复现和CI不稳定。FakeLLM让你精确安排"先read_file，再replace_text，最后结束"等回合，因此能测试每个边界分支。

阅读顺序

1. `tests/fakes.py`：FakeLLM如何保存请求和弹出预设回复。
2. `test_repair_agent.py`：工具循环、未知工具和调用上限。
3. `test_coordinator.py`：成功、重试、策略拒绝、Review拒绝和回归。
4. `test_live_deepseek.py`：为什么有 `@pytest.mark.live`。
5. `pyproject.toml`：默认 `-m 'not live'` 如何跳过付费测试。

常用命令

```
# 默认离线，不需要API Key
.\.venv\Scripts\python.exe -m pytest -q

# 只运行真实兼容性测试
.\.venv\Scripts\python.exe -m pytest -m live tests/test_live_deepseek.py -v

# 查看覆盖率
.\.venv\Scripts\python.exe -m pytest --cov=pyrepair --cov-report=term-missing -q
```

覆盖率表示哪些代码行被执行过，不等于测试质量。一个没有断言的测试也可能提升覆盖率；因此需要同时看断言是否验证了重要行为。

动手任务

挑选 `test_coordinator_generates_patch_without_touching_original`，回答：FakeLLM安排了哪些轮次？原项目不变由哪个断言证明？成功由哪些门共同证明？然后独立新增一个本地练习测试（可以不提交），让FakeLLM返回未知工具并观察结构化错误。

自测问题

1. FakeLLM验证的是模型智能吗?
2. 为什么live测试默认必须跳过?
3. Benchmark与单元测试有什么根本区别?
4. 91%覆盖率能证明Agent修复准确率是91%吗?

完成标准

你能明确说出：离线测试保证控制逻辑确定，live smoke保证API兼容，benchmark衡量固定任务端到端效果；三者缺一不可，但不能混称准确率。

学习记录：今天我能用FakeLLM稳定复现的异常分支是 _____。

第9天：Benchmark与数据表达

今日目标

学会运行、续跑和解释15例benchmark；能诚实地把结果写进简历和面试回答，不夸大模型能力。

建议时间分配

60分钟理解数据集与防泄漏；75分钟读runner和报告；75分钟分析正式JSONL；60分钟计算指标含义；45分钟写结果总结；30分钟自测。

数据集构成

固定任务包含条件、边界、异常、状态和类型转换5类，共15个独立单Bug pytest项目。每个case在运行前必须恰好有一个失败测试，且隐藏元数据不会复制给Agent，避免答案泄漏。

运行命令

```
# 运行全部case
.\.venv\Scripts\python.exe benchmarks/run_benchmark.py --model deepseek-v4-flash

# 只运行一个case
.\.venv\Scripts\python.exe benchmarks/run_benchmark.py --case case_01 --model deepseek-v4-
flash

# 从JSONL续跑未完成或技术错误case
.\.venv\Scripts\python.exe benchmarks/run_benchmark.py `
  --model deepseek-v4-flash `
  --resume "benchmarks\results\你的结果文件.jsonl"
```

PowerShell行尾反引号表示命令还没有结束，下一行继续。`--resume` 会读取已有JSONL，避免重复支付已经成功案例的费用。

当前正式结果

指标	DeepSeek V4 Flash
端到端成功	13 / 15
成功率	86.7%
首轮成功率	80.0%
平均耗时	17.50秒
Prompt / Completion Token	109,134 / 16,741
新增回归	0
Policy拒绝	0

86.7%是整个系统的端到端成功率，包含pytest运行、解析、模型修复、Policy、Review和验证，不是DeepSeek模型单独的“准确率”。

两个失败为什么值得保留

`case_05` 和 `case_08` 因pytest 9输出了无错误后缀的 `FAILED <node>` 摘要，当前解析器在调用LLM前停止。保留失败能说明：

- 评测没有为了好看删除坏结果。
- 系统瓶颈不一定在模型，也可能在解析基础设施。
- 下一项改进有明确证据，而不是凭感觉增加功能。

动手任务

阅读 `docs/benchmark-results.md` 和正式JSONL。任选一个成功case和一个失败case，对比字段：attempts、duration、tokens、regressions、policy_rejections、reason。写一段不超过120字的结果总结。

自测问题

1. 首轮成功率为什么低于最终成功率？
2. 为什么benchmark要固定case，而不是随便找项目？
3. 0新增回归说明了什么，不能说明什么？
4. 换成V4 Pro后为什么必须重新跑同一数据集？

完成标准

你可以在面试中准确表达“15个固定单Bug任务、13个端到端成功、两个失败发生在LLM调用前”，并主动说明样本量和任务范围限制。

学习记录：我会用一句话这样解释86.7%：_____。

第10天：亲手完成pytest 9解析改进

今日目标

把项目真正变成“自己的”：依据真实失败写回归测试，修改解析器，跑完整回归，再重新评测case05和case08。今天重点是方法，不是让AI直接替你提交答案。

建议时间分配

45分钟复现并缩小输出；60分钟写失败测试；90分钟做最小实现；60分钟跑全量回归；60分钟重跑case；45分钟整理前后证据；30分钟模拟面试说明。

改进目标

让解析器同时支持：

```
FAILED tests/test_x.py::test_name - AssertionError: detail
FAILED tests/test_x.py::test_name
```

但不能把普通日志中的任意 `FAILED` 单词误当成node id。

推荐TDD步骤

1. 从case05或case08保存一段最小真实pytest输出到fixtures。

2. 在 `test_pytest_parser.py` 新增测试，断言能提取node id。
3. 先运行新测试，确认它因为当前解析器能力不足而失败。
4. 只修改 `pytest_parser.py` 的最小正则或提取逻辑。
5. 运行解析器测试，确认新旧格式都通过。
6. 运行完整离线测试和Ruff，确认没有回归。
7. 只重跑case05与case08，保存新结果。
8. 再决定是否跑完整15例，比较端到端成功率。

建议命令

```
git switch -c fix/pytest9-failed-summary

.\.venv\Scripts\python.exe -m pytest `
  tests/test_pytest_parser.py::你的新测试名 -v

.\.venv\Scripts\python.exe -m pytest tests/test_pytest_parser.py -v
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m ruff check src tests benchmarks

.\.venv\Scripts\python.exe benchmarks/run_benchmark.py `
  --case case_05 --model deepseek-v4-flash
```

修改前必须回答

- 新格式中node id的边界在哪里结束?
- 参数化测试的 `[...]` 是否要保留?
- collection error没有 `FAILED` 摘要时怎么办?
- 如果一份输出有多个失败，MVP选择第一个还是全部?
- `extract_failed_node_ids()` 与 `parse()` 是否要复用同一规则?

修改后证据模板

```
问题：pytest 9无后缀FAILED摘要无法解析，2/15在LLM前失败。
测试：新增真实输出fixture和回归测试，修改前稳定失败。
实现：扩展node id提取规则，保留旧格式与参数化id支持。
回归：完整离线测试通过，Ruff通过。
评测：case05/case08重新运行，记录成功或新的失败层级。
结论：用真实benchmark驱动基础设施改进，而不是调整提示词凑结果。
```

自测问题

1. 为什么必须先看到新测试失败?
2. 如果case05修好了但case08仍失败，能否宣称15/15?
3. 为什么不能只改benchmark报告里的数字?
4. 这项改进为什么比再加一个复杂Agent更适合MVP?

完成标准

你能展示一次完整的“失败证据 -> 回归测试 -> 最小实现 -> 全量回归 -> benchmark对比”，并能明确说明结果是否真的提升。

学习记录：修改前证据是 _____；修改后证据是 _____；仍未解决的是 _____。

核心模块阅读地图

按调用链而不是文件名阅读

顺序	模块	核心入口	你必须回答的问题
1	`cli.py`	`fix()`	参数如何变成请求，退出码如何决定？
2	`config.py`	`load_config()`	模型和API Key何时校验？
3	`models.py`	多个dataclass	模块之间传递哪些稳定数据？
4	`coordinator.py`	`RepairCoordinator.run()`	阶段如何连接，何时提前退出？
5	`testing.py`	`TestRunner.run()`	如何安全启动和限制pytest？
6	`pytest_parser.py`	`PytestParser.parse()`	文本如何保守结构化？
7	`workspace.py`	`WorkspaceManager.create()`	原仓库为何不会被修改？
8	`tools.py`	`ToolRegistry`	Agent拥有哪些最小能力？
9	`repair_agent.py`	`RepairAgent.run()`	Tool Calling循环如何停止？
10	`deepseek.py`	`DeepSeekClient.complete()`	SDK响应如何变成ModelTurn？
11	`policy.py`	`PatchPolicyGate.check()`	哪些风险由确定性规则拦截？
12	`review_agent.py`	`ReviewAgent.review()`	独立只读审查如何结构化？
13	`verifier.py`	`Verifier.verify()`	目标与全量测试如何共同判定？
14	`recorder.py`	`RunRecorder`	如何留下可审计证据并脱敏？
15	benchmark runner	`run_selected_cases()`	指标如何从每个case聚合？

阅读一个函数的六步法

1. 只看签名和返回类型，不看函数体。
2. 找直接调用它的位置，明确调用者期望。
3. 找对应测试名，把测试名翻译为业务规则。
4. 阅读主路径，暂时折叠异常分支。
5. 再读异常分支，记录错误如何转换。
6. 用一个具体输入手算输出，再运行测试验证。

学习提示

如果一个函数看了20分钟仍然混乱，先回到它的数据对象和测试。通常不是Python语法难，而是你还不知道输入输出在整个流程中的位置。

Git最小知识：只学本项目立即需要的

四个概念

- 工作区：磁盘上正在编辑的文件。
- 暂存区：下一次Commit准备包含的变化。
- Commit：一次带说明的本地快照。
- Branch：指向一串Commit的可移动名字，用来隔离改进。

每次小改进的安全流程

```
# 1. 确认当前状态
git status
git branch --show-current

# 2. 从main创建本地分支
git switch -c fix/pytest9-failed-summary

# 3. 编辑并验证后查看差异
git diff

# 4. 只暂存本次相关文件
git add src/pyrepair/pytest_parser.py
git add tests/test_pytest_parser.py
git add tests/fixtures/pytest_outputs/pytest9-summary.txt

# 5. 创建本地快照
git commit -m "fix: parse pytest 9 failed summaries"
```

合并回main

```
git switch main
git merge fix/pytest9-failed-summary

# 合并后重新验证，再删除已合并分支
git branch -d fix/pytest9-failed-summary
```

`git add`不是上传；`git commit`也不是上传。只有配置远程仓库后执行 `git push` 才会把Commit发送到GitHub。Pull Request是GitHub上的协作审查流程，本地学习阶段可以后学。

禁止养成的习惯

- 不看 `git status` 就批量 `git add .`。
- 测试没跑就Commit“应该能用”的代码。
- 把API Key、`.env`、虚拟环境或运行产物提交进去。
- 遇到问题就使用 `git reset --hard`；它可能丢失未提交工作。
- 一个Commit同时混入功能、格式化和无关文档修改。

Git自测

1. Branch和worktree有什么区别？
2. `git diff`默认显示工作区相对暂存区的什么变化？
3. 为什么合并后还要在main重跑测试？

4. 本地Commit为什么不会自动出现在GitHub?

面试讲解模板

30秒版本

“PyRepair Agent是我为Python/pytest项目实现的安全候选补丁CLI。它先在临时副本复现并解析失败，再让DeepSeek通过五个受限工具提出最小源码修改，之后由确定性Policy、独立只读Review和目标加全量测试共同验证。它不会直接改原仓库，也不会自动Commit。15个固定单Bug任务中，Flash端到端成功13个，成功率86.7%，两个失败发生在pytest 9输出解析层。”

2分钟版本结构

1. 场景：通用代码Agent能力强，但自动修改代码需要更清晰的权限和验收边界。
2. 方案：聚焦pytest单Bug、小补丁，采用临时工作区和五个受限工具。
3. 闭环：baseline、Repair、Policy、Review、目标测试、全量回归、产物记录。
4. 工程点：FakeLLM离线测试、真实live smoke、固定benchmark、断点续跑和脱敏。
5. 结果：13/15端到端成功，0新增回归；两个失败暴露pytest 9解析缺口。
6. 下一步：先修解析器并用同一数据集重测，再考虑多失败聚类和安全应用补丁。

“和Claude Code相比有什么优势”

不要说PyRepair比Claude Code更强。更好的回答是:

“Claude Code是通用编程Agent，任务范围和工具能力更广；PyRepair的价值是把问题缩小到pytest自动修复，并把可修改路径、补丁规模、独立Review、目标测试、全量回归和可审计产物做成明确协议。它的优势不是能力广度，而是在这个窄场景中更容易复现、量化和接入自动化。”

高频追问与回答要点

追问	回答要点
为什么不用任意Shell?	防止命令注入、越权、删除文件和绕过固定测试入口
为什么不让模型判定成功?	模型有概率性，成功必须由Policy和pytest决定
Review也是LLM，可靠吗?	Review处理语义风险，硬边界仍由Python规则，最终还有真实测试
为什么普通测试不用LLM?	稳定、离线、无费用，可精确覆盖异常分支
成功率样本是否太小?	是，15例只支持MVP范围内结论，因此明确样本与类型，不外推
如果换Pro会怎样?	保持同一数据集和配置重新测，不能先声称更好
最大局限是什么?	单失败、小补丁、解析器格式覆盖有限、仅精确文本替换

简历写法与证据对应

推荐项目描述

“独立开发PyRepair Agent：基于DeepSeek V4 Tool Calling构建pytest故障诊断与候选补丁闭环，通过临时工作区、确定性Patch策略、独立Review与全量回归限制高风险修改；建立15个真实Bug任务的可复现评测，Flash端到端成功率86.7%，首轮成功率80%，0新增回归。”

每一句都必须能展示证据

简历表述	证据位置
Tool Calling闭环	`repair_agent.py`、`deepseek.py`、相关测试
临时工作区	`workspace.py`、原项目不变测试
确定性Patch策略	`policy.py` 与参数化安全测试
独立Review	`review_agent.py` 与新上下文测试
全量回归	`verifier.py` 与新增回归测试
86.7%	benchmark Markdown和正式JSONL
0新增回归	15例记录中的regressions指标

不要这样写

- “实现全自动软件工程Agent”--项目没有Issue理解、Commit、Push和PR闭环。
- “模型准确率86.7%”--这是整个系统在固定任务上的端到端成功率。
- “完全安全”--安全边界能降低风险，但候选补丁仍需开发者审查。
- “支持所有pytest错误”--当前有明确的pytest 9摘要格式缺口。

面试提示

面试官不怕你项目有局限，怕你不知道局限。主动说明边界、失败数据和下一步实验，通常比夸大功能更有说服力。

常见错误与排查手册

找不到pyrepair命令

检查是否使用了正确虚拟环境：

```
.\.venv\Scripts\python.exe -m pip show pyrepair-agent
.\.venv\Scripts\pyrepair.exe --help
```

如果第一条找不到包，在项目根目录重新运行 `-m pip install -e ".[dev]"`。

pytest出现import file mismatch

先检查仓库根目录下是否同时存在一个包含同名tests的 `.worktrees`。pytest从根目录自动递归时可能收集两套同名模块。明确运行 `pytest tests`只能作为诊断；正确收尾通常是移除已经合并的重复worktree，或在pytest配置中合理排除工作树目录。

`.pytest_cache`拒绝访问

这是缓存目录权限问题，不代表测试断言失败。可以加 `-p no:cacheprovider` 临时禁用cache provider，或在确认路径后修复/删除该缓存。不要因为缓存警告忽略真正的测试失败摘要。

API请求失败

按顺序检查：环境变量是否在当前PowerShell会话、模型名是否允许、网络/代理、HTTP状态码、SDK重试后的最终异常。任何输出都不能打印完整API Key。

运行成功但原项目没有变化

这是预期行为。PyRepair只生成候选补丁，检查最新运行目录中的 `fix.patch`。MVP不自动应用补丁。

benchmark第一次很慢

记录阶段和耗时，检查网络、模型服务与重试。使用JSONL `--resume` 避免重新跑成功case。不要手工删除失败记录后只展示好结果。

Debug顺序

```
先复现 -> 确认阶段 -> 阅读report -> 阅读pytest.log
-> 沿trace还原工具调用 -> 最后才修改代码或提示词
```

十天验收清单

使用能力

- ☐ 能重建虚拟环境并解释每条命令。
- ☐ 能运行离线测试、覆盖率和Ruff。
- ☐ 能运行dry-run与真实Flash调用。
- ☐ 能找到最新产物目录并阅读四个文件。

架构能力

- ☐ 能不看图画出完整调用链。
- ☐ 能说明Coordinator与各协作者的边界。
- ☐ 能说出五个工具和每个限制。
- ☐ 能区分Policy、Review、Verifier。
- ☐ 能解释目标测试与全量回归。

测试与评测能力

- ☐ 能说明FakeLLM、live smoke与benchmark的区别。
- ☐ 能解释13/15与86.7%的准确含义。
- ☐ 能说明case05、case08失败层级。
- ☐ 能根据report和trace定位一次失败。

开发与面试能力

- ☐ 能创建本地分支并提交一个小改进。
- ☐ 能使用TDD完成pytest 9解析改进。
- ☐ 能做30秒和2分钟项目介绍。
- ☐ 能回答“和Claude Code相比有什么价值”。
- ☐ 能主动说明项目局限和下一步实验。

最终模拟验收

请朋友或对着录音完成一次15分钟模拟：3分钟演示、5分钟架构讲解、5分钟追问、2分钟总结。如果某个问题只能说“代码里是这样写的”，就回到对应模块，用输入、输出、必要性和失败处理重新学习。

后续30天进阶路线

第1周：修复基础设施缺口

完成pytest 9摘要支持；增加collection error和参数化node id样本；对case05、case08及完整15例重测。目标是学会用评测定位瓶颈，而不是无目的增加Agent能力。

第2周：安全应用候选补丁

设计 `pyrepair apply <run-id>`：默认预览、要求用户确认、验证原仓库状态未漂移、应用后再跑测试。不要一开始自动Commit或Push。

第3周：多失败诊断

研究多个失败node id如何聚类，区分同一根因和多个独立Bug。先做只读诊断报告，再决定是否允许逐项修复。

第4周：GitHub Actions手动演示

先用 `workflow_dispatch` 手动触发，在CI中运行pytest与PyRepair并上传候选补丁产物。稳定后再评估pytest失败自动触发。自动化不等于自动合并，仍应保留人工审查。

进阶学习主题优先级

优先级	主题	原因
1	pytest输出、subprocess、路径安全	直接决定系统可靠性
2	Tool Calling消息协议和结构化输出	直接决定Agent循环
3	Git branch、Commit、Actions产物	直接决定工程展示
4	Pydantic或更强JSON校验	数据对象复杂后再引入
5	多Agent框架、RAG、长期记忆	当前MVP不需要，后学

结语

真正“拥有”一个项目，不是能背出所有代码，而是你能解释为什么这样设计、能用证据判断哪里失败、能做一个小改进并证明没有破坏其他功能。沿着这十天路线完成一次完整改进后，PyRepair就不再只是你运行过的项目，而会成为你能负责、能迭代、能在面试中经得起追问的工程作品。

结课行动卡

- 完成一次不看提示的CLI演示，并保存四类运行产物。
- 画一张自己的架构图，用5分钟讲清数据流和三层安全门。
- 独立分析一个失败case，指出失败层级与下一项最小实验。
- 用TDD完成pytest 9解析改进，并用回归与benchmark证明结果。

学习目标

学习完成的标志不是“所有文件都看过”，而是上面四项都有可以展示的真实证据。把证据整理到项目README、演示脚本和面试笔记中，你就拥有了一套可复现的项目故事。